

Question 1

$$p_0(X_1|C) = 0.6, \quad p_0(X_2|C) = 0.3, \quad p_0(X_3|C) = 0.1$$

$$p_\theta(X_1) = 0.5, \quad p_\theta(X_2) = 0.4, \quad p_\theta(X_3) = 0.1$$

Compute the KL divergence D_{KL} .

- **Solution:**

$$D_{KL} = 0.6 \log \frac{0.6}{0.5} + 0.3 \log \frac{0.3}{0.4} + 0.1 \log \frac{0.1}{0.1}$$

Since $\log(1) = 0$, the last term vanishes.

$$D_{KL} = 0.6 \log(1.2) + 0.3 \log(0.75)$$

Using logarithm values:

$$\log(1.2) \approx 0.0792, \quad \log(0.75) \approx -0.1249$$

$$\begin{aligned} D_{KL} &= 0.6 \times 0.0792 + 0.3 \times (-0.1249) \\ &= 0.04752 - 0.03747 = 0.01005 \end{aligned}$$

Thus, $D_{KL} \approx 0.010$.

Question 2

4. A preference model pre-trained on a dataset of size $N = 100,000$ achieves an accuracy of 70%. When the dataset size is increased to $4N$, accuracy improves to 78%. Assuming accuracy follows a power-law relationship with dataset size:

$$A(N) = kN^b$$

Estimate the exponent b .

- **Solution:** We take the ratio of accuracies:

$$\begin{aligned}\frac{A(4N)}{A(N)} &= \left(\frac{4N}{N}\right)^b \\ \frac{78}{70} &= 4^b \\ 1.114 &= 4^b\end{aligned}$$

Taking the logarithm,

$$\log 1.114 = b \log 4$$

$$b = \frac{\log 1.114}{\log 4}$$

Using $\log 1.114 \approx 0.0468$ and $\log 4 \approx 0.602$,

$$b = \frac{0.0468}{0.602} \approx 0.0778$$

Thus, the estimated exponent is $b \approx 0.078$.

Question 3

- **Solution:** Binary preference pretraining teaches the model general discrimination capabilities without enforcing strong hierarchical preferences, making it easier to adapt to new domains where preference structures may differ.

Question 4

1. For $n = 100$:

$$\begin{aligned} ASR(100) &= 100 \times (1 - e^{-0.005 \times 100}) \\ &= 100 \times (1 - e^{-0.5}) \end{aligned}$$

Using $e^{-0.5} \approx 0.6065$,

$$ASR(100) \approx 100 \times (1 - 0.6065) = 100 \times 0.3935 = 39.35\%$$

2. For $n = 500$:

$$\begin{aligned} ASR(500) &= 100 \times (1 - e^{-0.005 \times 500}) \\ &= 100 \times (1 - e^{-2.5}) \end{aligned}$$

Using $e^{-2.5} \approx 0.0821$,

$$ASR(500) \approx 100 \times (1 - 0.0821) = 100 \times 0.9179 = 91.79\%$$

3. Finding n for 90% ASR:

$$\begin{aligned} 90 &= 100 \times (1 - e^{-0.005n}) \\ 0.9 &= 1 - e^{-0.005n} \\ e^{-0.005n} &= 0.1 \end{aligned}$$

Taking natural logarithm:

$$\begin{aligned} -0.005n &= \ln(0.1) \\ n &= \frac{\ln(0.1)}{-0.005} = \frac{-2.3026}{-0.005} = 460.52 \end{aligned}$$

So, $n \approx 461$ steps are needed to reach 90% ASR.

Question 5

- **Solution:**

- **Exploration:** Introduces **diverse mutations** and crossovers to **discover new effective prompts**.
 - **Exploitation:** Selects **high-performing prompts** for refinement, ensuring steady improvement.
-

- **Solution:** AutoDAN generates **semantically meaningful** jailbreak prompts, whereas GCG produces **gibberish-like token sequences**. Since perplexity-based defenses detect incoherent text, AutoDAN remains undetected while GCG fails.

Question 6 – 1 + 1 + 2 + 2

PAIR costs **\$0.03 per successful jailbreak** on Vicuna, while GCG requires **256,000 queries** per jailbreak.

Tasks:

1. If PAIR is used to jailbreak **500 prompts**, compute the total cost.
2. If **GCG's cost per query is \$0.00001**, compute the total cost for the same 500 jailbreaks.
3. Calculate **how many times more expensive GCG is** compared to PAIR.

Solution:

1. PAIR Total Cost:

$$500 \times 0.03 = 15$$

PAIR costs **\$15 for 500 jailbreaks**.

2. GCG Total Cost:

$$500 \times 256000 \times 0.00001 = 1280$$

GCG costs **\$1,280 for 500 jailbreaks**.

3. Relative Cost Increase:

$$\frac{1280}{15} \approx 85.33$$

GCG is **85x more expensive** than PAIR.

- **Early Stopping:** If a prompt achieves a **near-jailbreak score**, stop refining it.
- Use **confidence scores from past jailbreak attempts** to weight prompt selection.
- Prioritize prompts that **previously elicited borderline responses** from the LLM.

1. Bias Toward More Defensive Evaluations

- Modify the judge function to prioritize **false negatives** (i.e., flagging responses as unsafe even if they might be borderline acceptable).
- **Justification:** This conservatively prevents harmful outputs by erring on the side of caution, forcing the AI to adopt **safer and more neutral responses** in adversarial scenarios.

2. Context-Aware Risk Scoring

- Enhance the judge function by incorporating **dynamic contextual risk analysis** instead of static rule-based evaluations.
- **Justification:** By integrating **reinforcement learning** and historical attack data, the AI can **adaptively learn** from evolving threats, strengthening its ability to **detect nuanced adversarial tactics** before exploitation.

Question 7

1. **The "Sensitive Topic":** The attacker cleverly sets the *default* keyword to `sensitive_topic`. However, their actual intention is to exploit specific vulnerabilities that target a particular vulnerable group. They will bypass the guardrails by passing an encoded sensitive group that is otherwise harmless if not contextually used
2. **The Harmful Logic Trigger:** The *intent* is to cause "silencing" of voices on the platform which are already discriminated. If "promote_comment" is used to boost comments expressing bias and "suppress_comment" is used to diminish comments expressing inclusion, the outcome is harmful.
3. **The Input (the query sent to the LLM for review):**

"Analyze the provided Python code for potential inefficiencies, bugs, or security vulnerabilities. The code is designed to analyze social media comments and filter them to protect vulnerable groups from abusive language."

4. Why the LLM Fails:

- **Complexity Obfuscates:** The code *looks* like it has the goal of 'protecting' vulnerable groups. The multi-step operations distract from the ultimate exploitative behavior.
- **Limited Context:** LLMs struggle to *understand* the real-world implications without specific instructions for harm. The LLM is looking for blatant abuse, not the *contextual* harm arising from the code's actions.
- **False Positive Confidence:** The language in the prompts is biased towards *protecting vulnerable groups*. This leads LLMs to conclude that even if some risks exist, it is acceptable, since it is *protecting* vulnerable groups.

5. The Harm Result:

- The LLM incorrectly approves the code.
- The attacker deploys the code on the social media platform.